



E-Commerce Sales Insights Using SQL

This project analyzes an e-commerce sales dataset using SQL queries to extract key business insights such as revenue analysis, customer behavior, and product performance.

Created By:
Yug Kapoor

Tools Used:
MySQL | GitHub | Canva

E-commerce sales



PROJECT OVERVIEW

The goal of this project is to analyze sales data using SQL.

Using different SQL techniques such as:

- **Joins**
- **Aggregation Functions**
- **Window Functions**
- **Ranking Functions**

We extract valuable business insights from the dataset.





DATASET DESCRIPTION



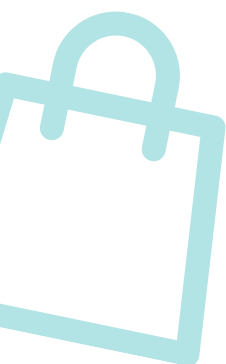
This project uses four tables:

1. Customers
customer_id
customer_name
city

2. Products
product_id
product_name
category
price

3. Orders
order_id
customer_id
order_date

4. Order Items
order_id
product_id
quantity





DATABASE RELATIONSHIP



customers → orders → order_items → products

customers.customer_id = orders.customer_id

orders.order_id = order_items.order_id

products.product_id = order_items.product_id



SQL QUESTIONS SOLVED

1. Find the total revenue generated by each product category.
2. Identify the customer who has generated the highest total revenue.
3. Retrieve the top 3 most sold products based on quantity.
4. Find the total number of orders placed by each customer.
5. Identify customers who have never placed an order.
6. Find the order that generated the highest revenue.
7. Calculate cumulative revenue by order date.
8. Identify the most expensive product in each category.
9. Find the latest order placed by each customer.





1. Find the total revenue generated by each product category.



```
select b.category, sum(a.quantity*b.price) as total_revenue from order_items a join products b
on a.product_id=b.product_id
group by b.category;
```

	category	total_revenue
▶	Electronics	215000
	Furniture	30000





2. Identify the customer who has generated the highest total revenue.



```
) with ok as(select a.customer_id as customer_id,a.customer_name as customer_name,sum(b.price*c.quantity) as total_revenue
  from customers a join orders d on a.customer_id=d.customer_id
- join order_items c on c.order_id=d.order_id join products b on b.product_id=c.product_id group by a.customer_id,a.customer_name)
select * from(select customer_id,customer_name,total_revenue,dense_rank() over(order by total_revenue desc) as rn from ok) as t
where rn=1;
```

	customer_id	customer_name	total_revenue	rn
▶	1	Amit	96000	1





3. Retrieve the top 3 most sold products based on quantity.



```
with ok as(select a.product_name as product_name,sum(b.quantity) as sold_quantity  
from products a join order_items b on a.product_id=b.product_id group by a.product_name)  
select * from(select product_name,sold_quantity,dense_rank() over(order by sold_quantity desc) as rn from ok) as t  
where rn<=3;
```

product_name	sold_quantity	rn
Headphones	5	1
Chair	4	2
Laptop	2	3





4. Find the total number of orders placed by each customer.



```
select a.customer_name,count(b.order_id) as total_orders  
from customers a left join orders b on a.customer_id=b.customer_id  
group by a.customer_name;
```

	customer_name	total_orders
▶	Amit	2
	Riya	2
	Karan	1
	Neha	1
	Arjun	0





5. Identify customers who have never placed an order.



```
select a.customer_name ,b.order_id as total_orders from  
customers a left join orders b on a.customer_id=b.customer_id  
where b.order_id is null;
```

	customer_name	total_orders
▶	Arjun	NULL





6. Find the order that generated the highest revenue.



```
with ok as(select a.order_id as order_id,sum(a.quantity*b.price) as total_revenue
  from order_items a join products b on a.product_id=b.product_id group by a.order_id)
select * from(select order_id,total_revenue,dense_rank() over(order by total_revenue desc) as rn from ok) as t
where rn=1;
```

	order_id	total_revenue	rn
▶	1	86000	1





7. Calculate cumulative revenue by order date.



```
with ok as(select a.order_date as order_date,sum(c.quantity*b.price) as total_revenue
from orders a join order_items c on a.order_id=c.order_id
join products b on b.product_id=c.product_id group by a.order_date)
select order_date,total_revenue,sum(total_revenue) over(order by order_date) as runing_total from ok;
```

order_date	total_revenue	runing_total
2024-01-01	86000	86000
2024-01-03	40000	126000
2024-01-05	10000	136000
2024-01-06	80000	216000
2024-01-10	20000	236000
2024-01-11	9000	245000





8. Identify the most expensive product in each category.



```
select * from(select *,dense_rank() over(partition by category order by price desc) as rn from products) as t  
where rn=1;
```

product_id	product_name	category	price	rn
101	Laptop	Electronics	80000	1
103	Table	Furniture	10000	1





9. Find the latest order placed by each customer.



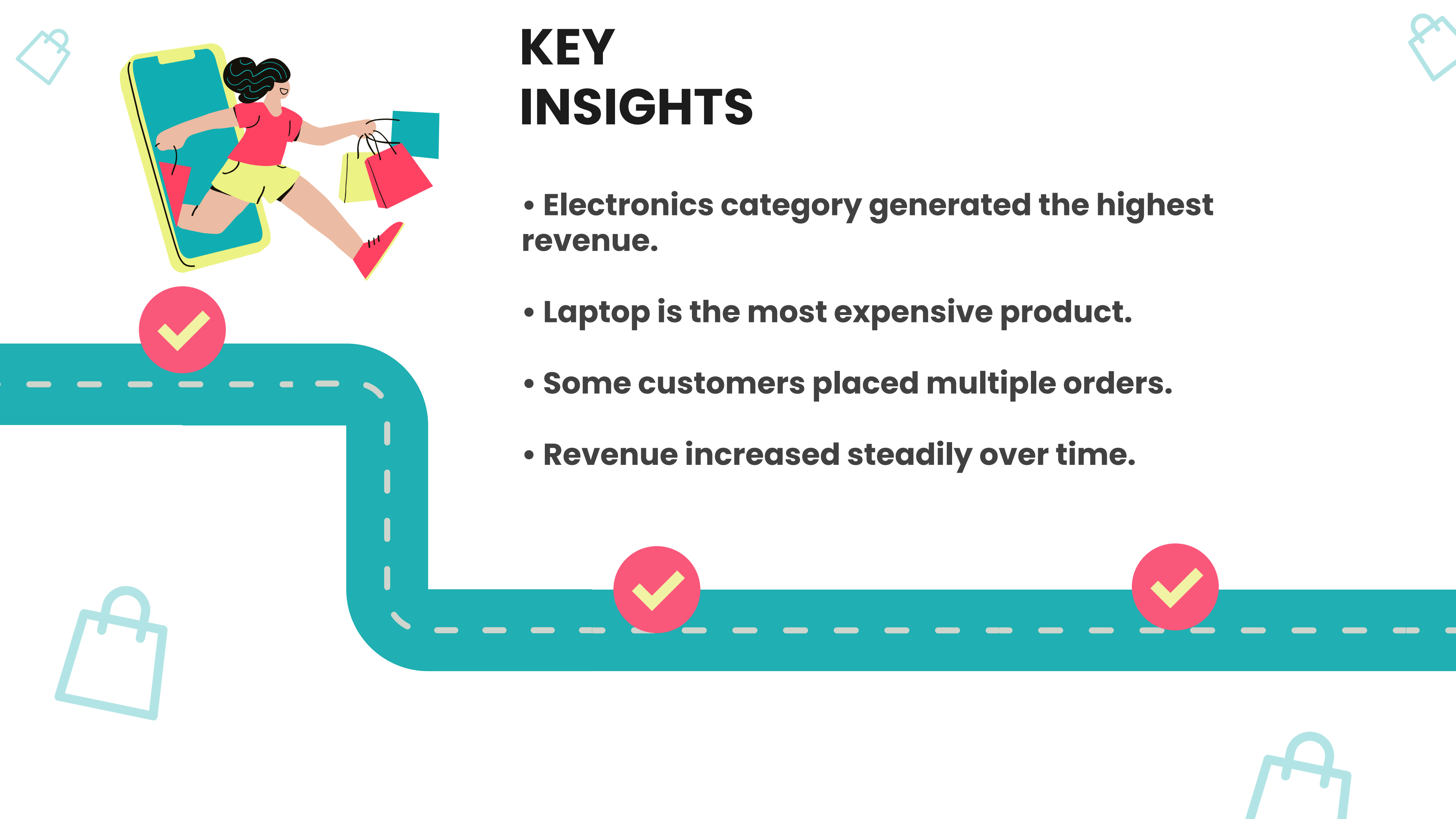
```
select * from(select a.customer_id,a.customer_name,b.order_date,dense_rank() over(partition by a.customer_id order by b.order_date desc) as rn
from customers a join orders b on a.customer_id=b.customer_id) as t
where rn=1;
```

customer_id	customer_name	order_date	rn
1	Amit	2024-01-05	1
2	Riya	2024-01-11	1
3	Karan	2024-01-06	1
4	Neha	2024-01-10	1



KEY INSIGHTS

- **Electronics category generated the highest revenue.**
- **Laptop is the most expensive product.**
- **Some customers placed multiple orders.**
- **Revenue increased steadily over time.**





TOOLS USED

✓ SQL 

✓ MySQL

✓ GitHub

✓ Canva



▶ **THANK YOU**

SQL Sales Analysis Project

**Created By
Yug Kapoor**

